Host guide: Smart-RUDP review

For the presenter only. Each page shows one slide of the real deck, what to say, and the question it is likely to bring.

Computer Networks | Faculty: Anuradha G | 9 Oct 2026

Plan for the session

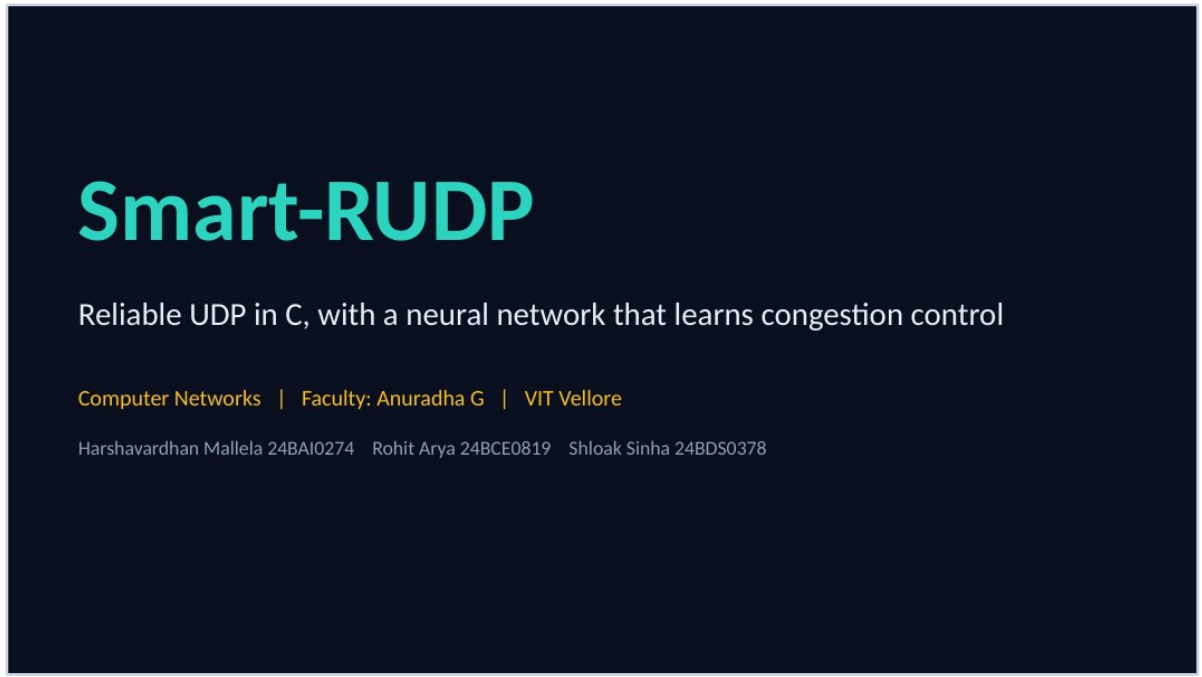
Total talking time about 15 minutes, then questions.

#	Slide	Time	#	Slide	Time
1	Title	0:20	21	How we tested	0:40
2	Why	0:40	22	Issues	0:50
3	What we built	0:40	23	Changes	1:00
4	Headline results	1:00	24	Limits	0:50
5	Up to 83% less time	0:40	25	Learned	0:50
6	Scorecard	1:00	26	Thank you	0:15
7	Real clock	1:00	27	Neural agent in the C transport	1:00
8	Window picture	0:40	28	Safety net: hand over to AIMD	0:50
9	Kernel	1:00	29	More evidence: baselines, SACK, reordering	1:00
10	Changing paths	1:00	30	Fairness, ablations, chat fixes	0:50
11	Fairness	1:00	31	Fairness and file transfer	0:50
12	Two machines	0:45	32	Closing gaps: fairness and cross traffic	0:50
13	RUDP Drop	1:00	33	Real sockets found what the emulator could not	0:50
14	Chat	1:00	34	Problems we hit and fixed	0:40
15	Packet Tracer	0:50	35	Our first learned agent: an honest result	0:50

Rules: say the limits before she asks; answer in one or two sentences; if you do not know, say what you would check.

1. Title

Time on this slide: 0:20



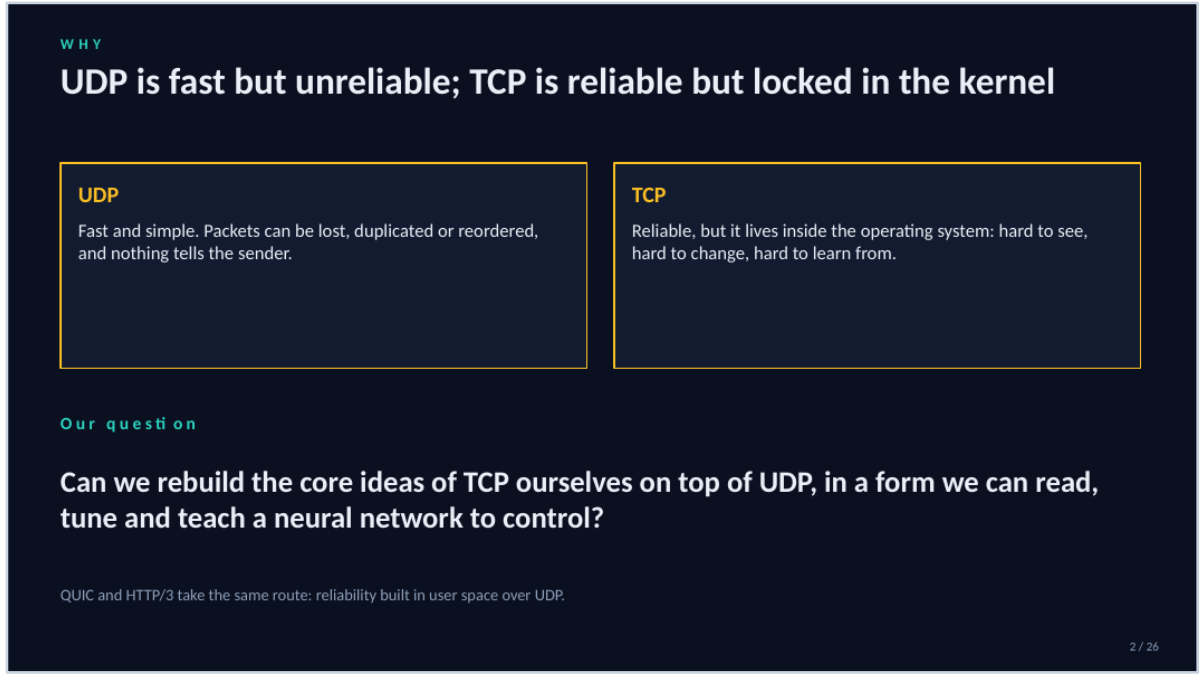
SAY

- We built RUDP, a reliable protocol on UDP in C, and taught a neural network to control its congestion window.

IF ASKED Who did what? Harsh: transport and agent; Rohit and Shloak: network design and tools.

2. Why

Time on this slide: 0:40



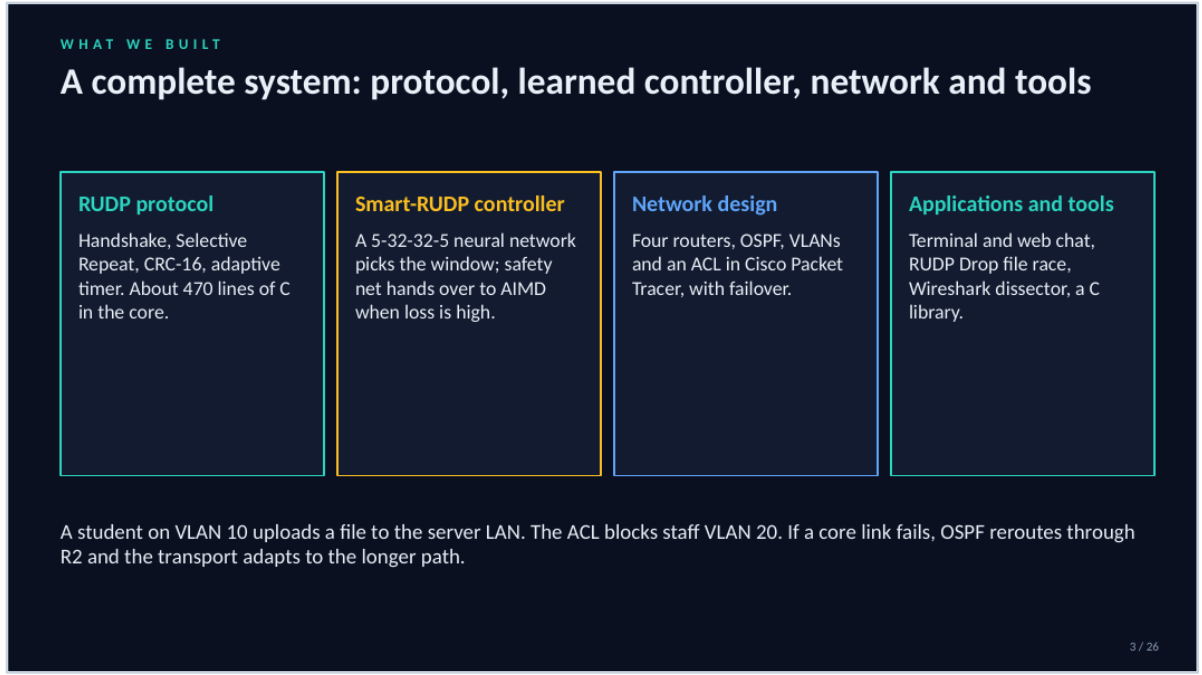
IF ASKED Is this like QUIC? Same idea: reliability in user space over UDP.

SAY

- UDP is fast but unreliable. TCP is reliable but sits in the kernel.
- Our question: can we rebuild TCP's core ideas ourselves, in a form we can read, tune and teach a network to control?

3. What we built

Time on this slide: 0:40



SAY

- Four parts: the protocol, the learned controller, the Packet Tracer network, and the tools (chat, Drop, Wireshark dissector).

IF ASKED How big is the core? About 470 lines of C.

4. Headline results

Time on this slide: 1:00



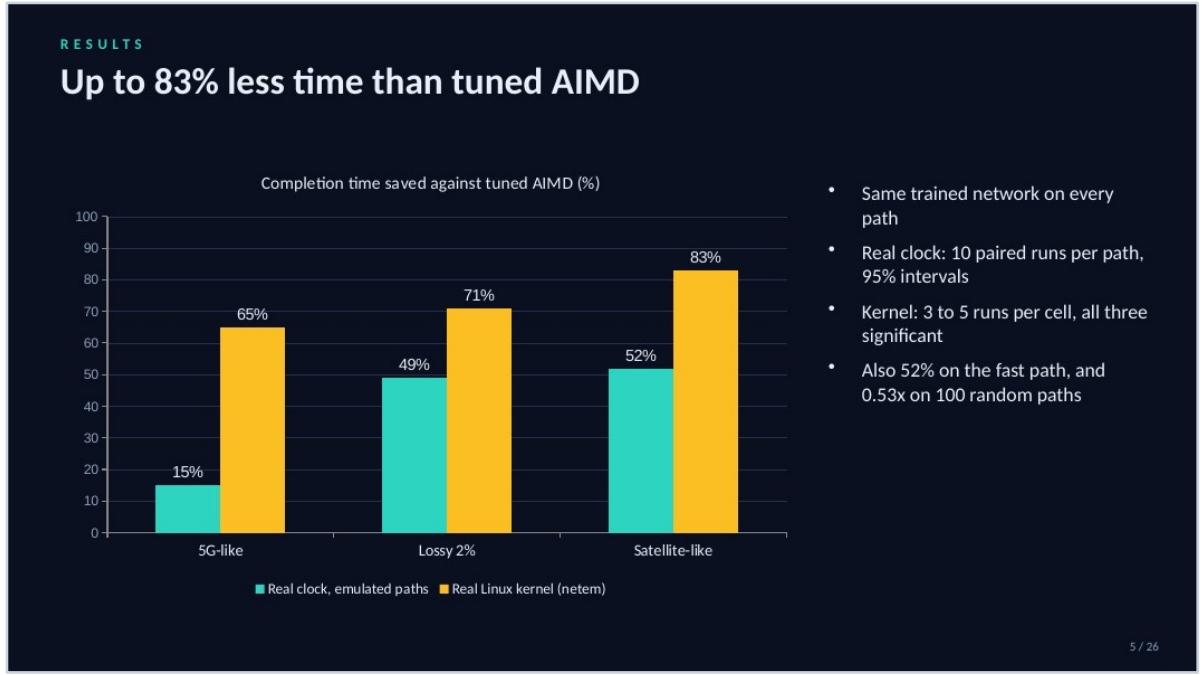
IF ASKED Against what? Tuned AIMD and tuned CUBIC, not weak baselines.

SAY

- Lead with the numbers: 15 to 52% less time on emulated paths, 65 to 83% on the real Linux kernel, 0.53x on 100 random unseen paths.
- Fastest in all four changing-path cases. Fairer than AIMD, and it takes 24 to 34% next to kernel TCP.

5. Up to 83% less time

Time on this slide: 0:40



IF ASKED Why is the kernel gain bigger? Tuned AIMD does badly with netem's real queues.

SAY

- Same trained network on every path, no retuning. Left bars real clock, right bars real kernel.

6. Scorecard

Time on this slide: 1:00



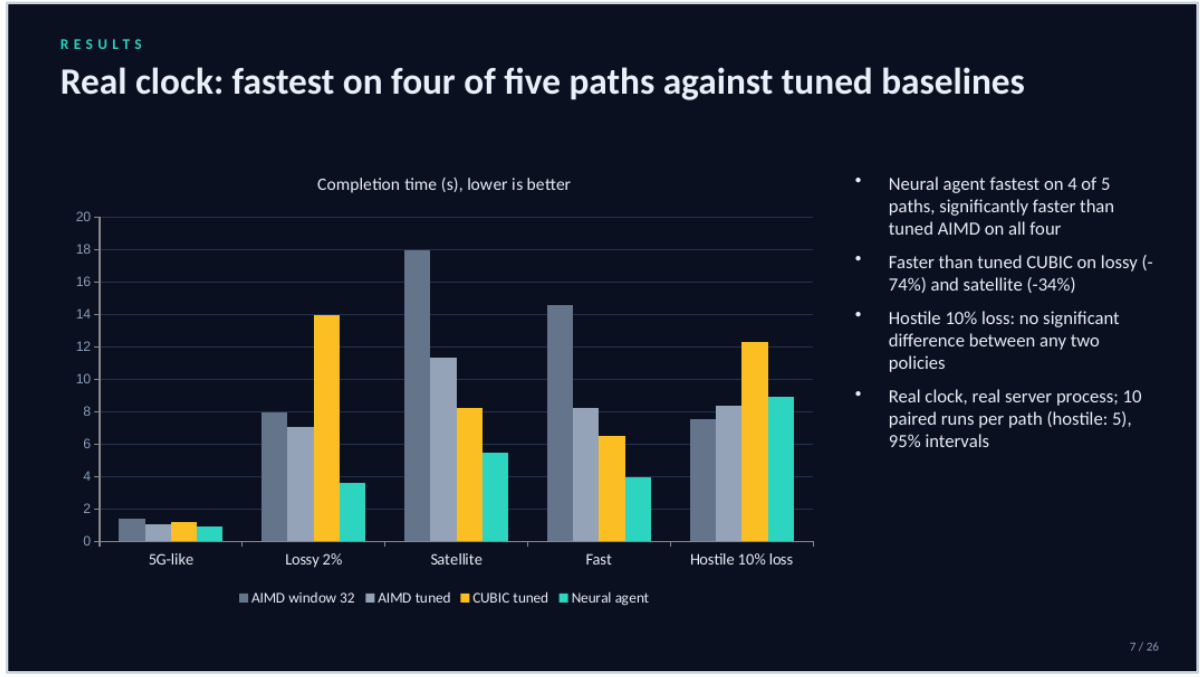
IF ASKED Did you run FTP? No; it runs on kernel TCP, so the TCP row is the comparison.

SAY

- One table of every alternative: AIMD, CUBIC, BBR-style, Q-table, plain UDP and kernel TCP.
- Honest row: kernel TCP is 1.5 to 1.7x faster than the agent. FTP and SCP run on TCP so that row stands for them.

7. Real clock

Time on this slide: 1:00



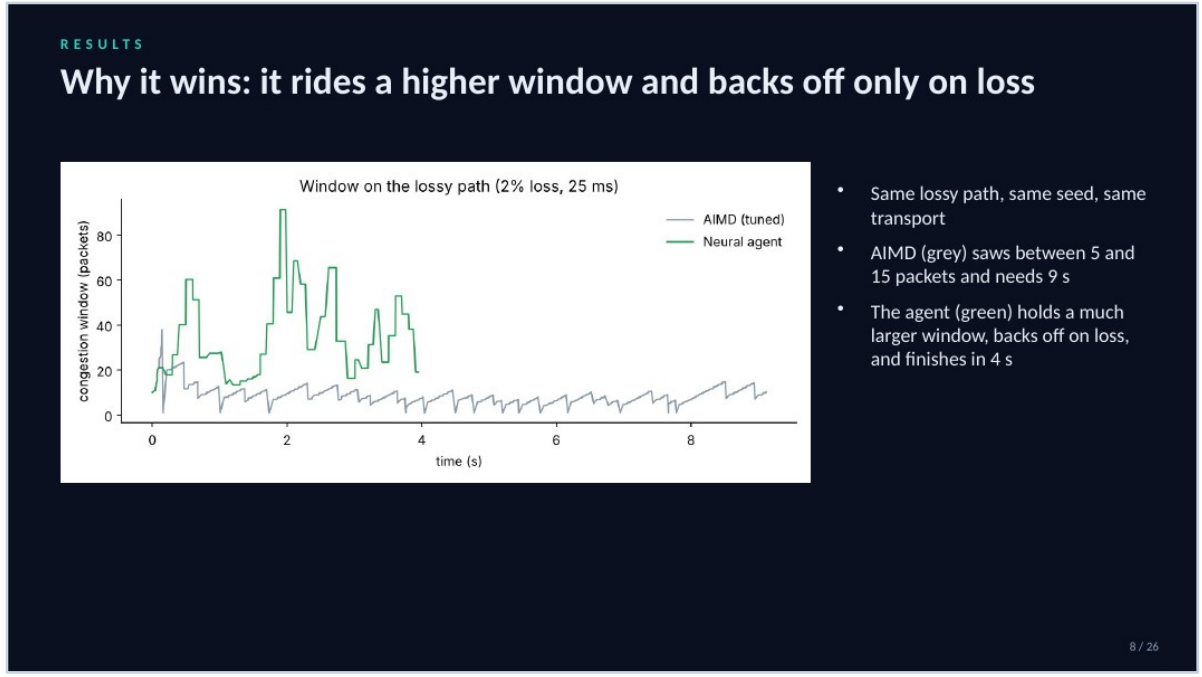
IF ASKED How many runs? 10 paired runs per path, 95% intervals.

SAY

- Fastest on four of five paths. Lossy 3.6 s against 7.1 s tuned AIMD and 13.9 s CUBIC.
- At 10% loss no policy is significantly different.

8. Window picture

Time on this slide: 0:40



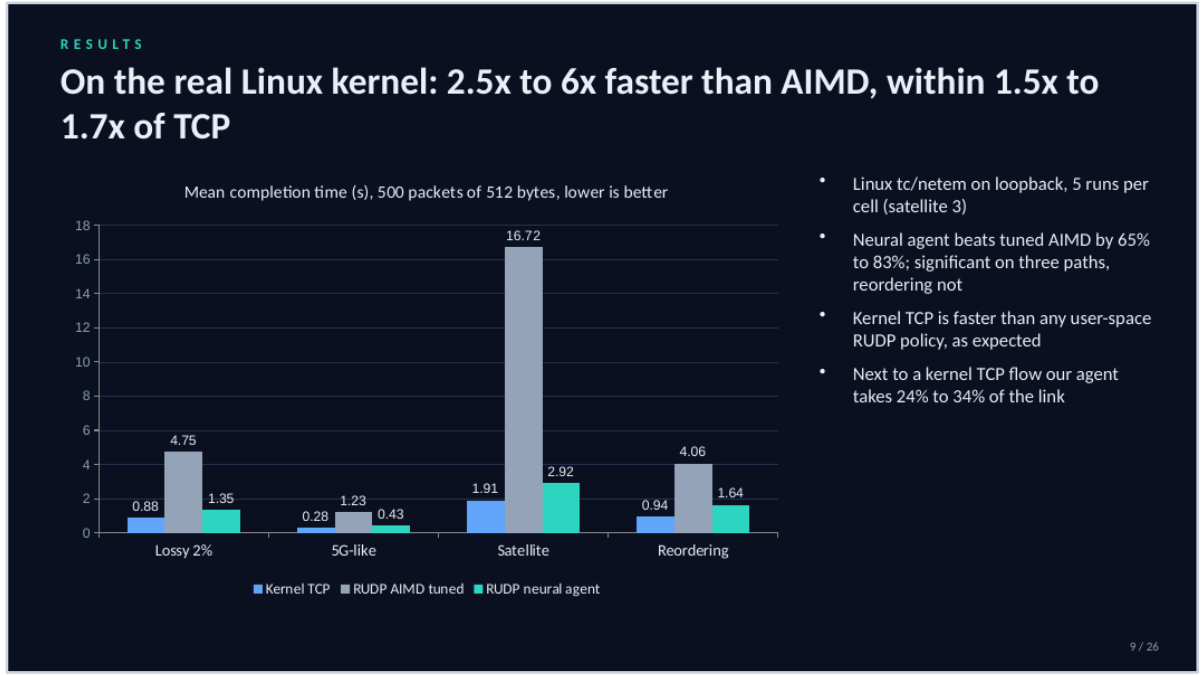
IF ASKED Why not just a bigger AIMD cap? We tuned it to 128; the agent still wins.

SAY

- Same path, same seed. AIMD saws between 5 and 15 packets and needs 9 s; the agent holds a much bigger window, backs off on loss, finishes in 4 s.

9. Kernel

Time on this slide: 1:00



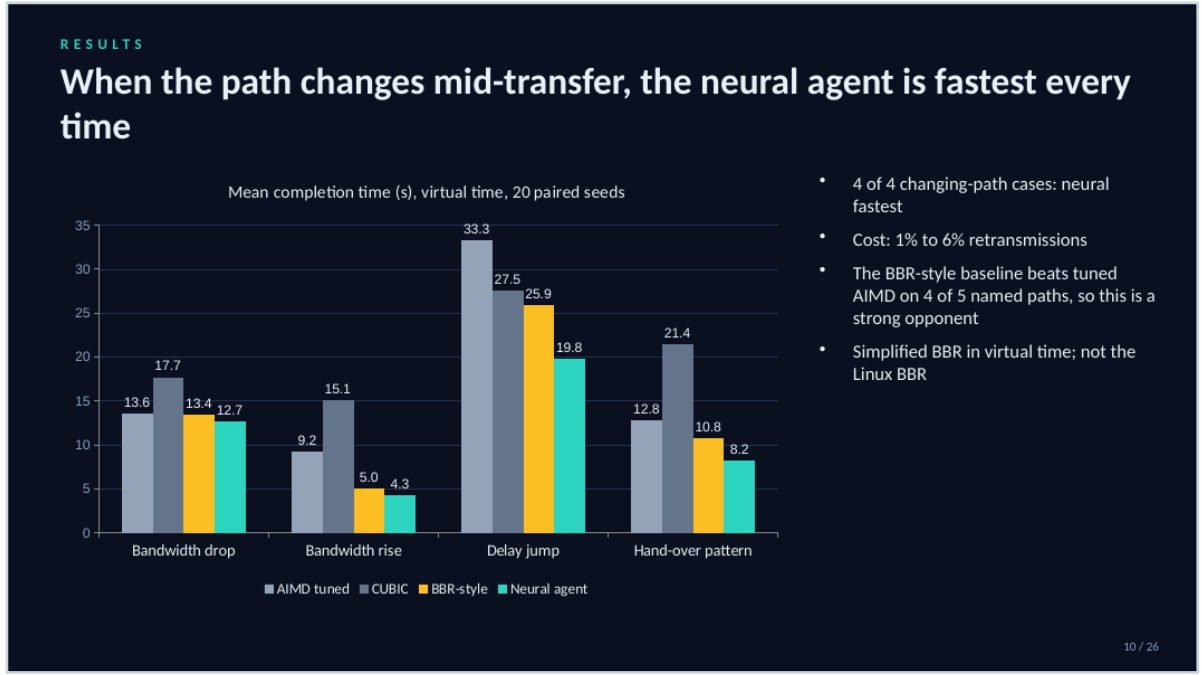
IF ASKED Why is TCP faster? In-kernel, years of tuning, no user-space overhead.

SAY

- Real Linux queues with netem, 500 real 512 byte packets.
- Agent beats tuned AIMD by 65 to 83%. Kernel TCP is still faster, as expected.

10. Changing paths

Time on this slide: 1:00



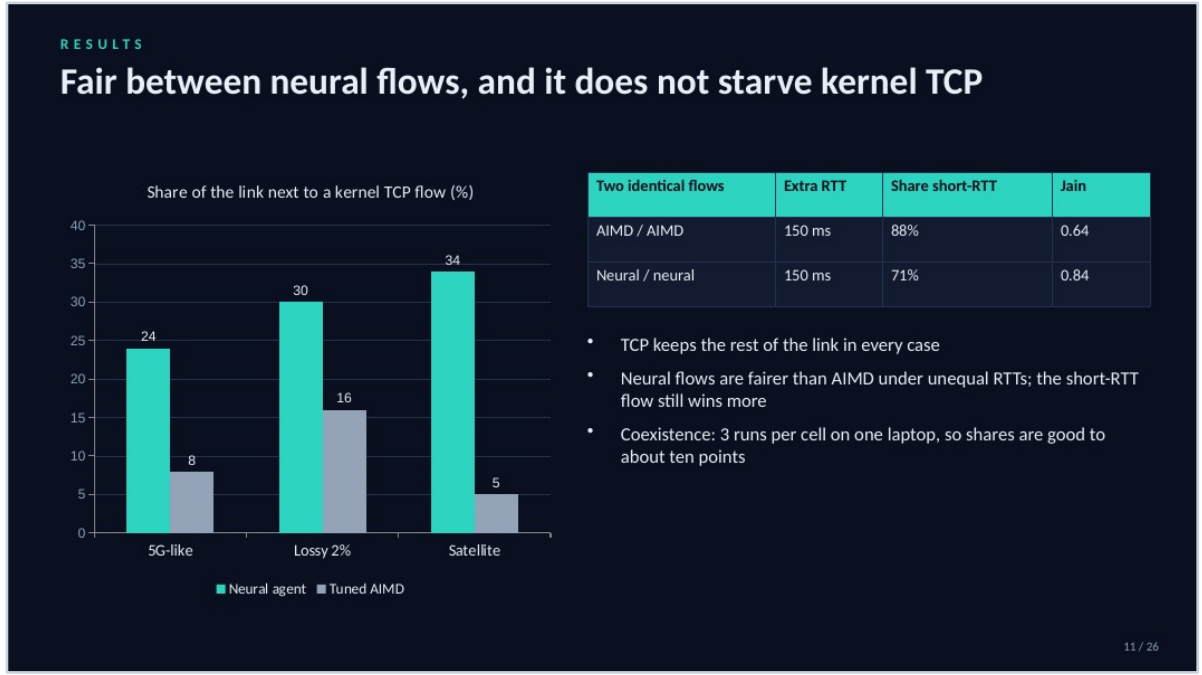
IF ASKED Is it real BBR? A simplified version in virtual time.

SAY

- Bandwidth drop, rise, delay jump, hand-over: the agent is fastest in all four.
- BBR-style is a strong opponent here and beats AIMD on most.

11. Fairness

Time on this slide: 1:00



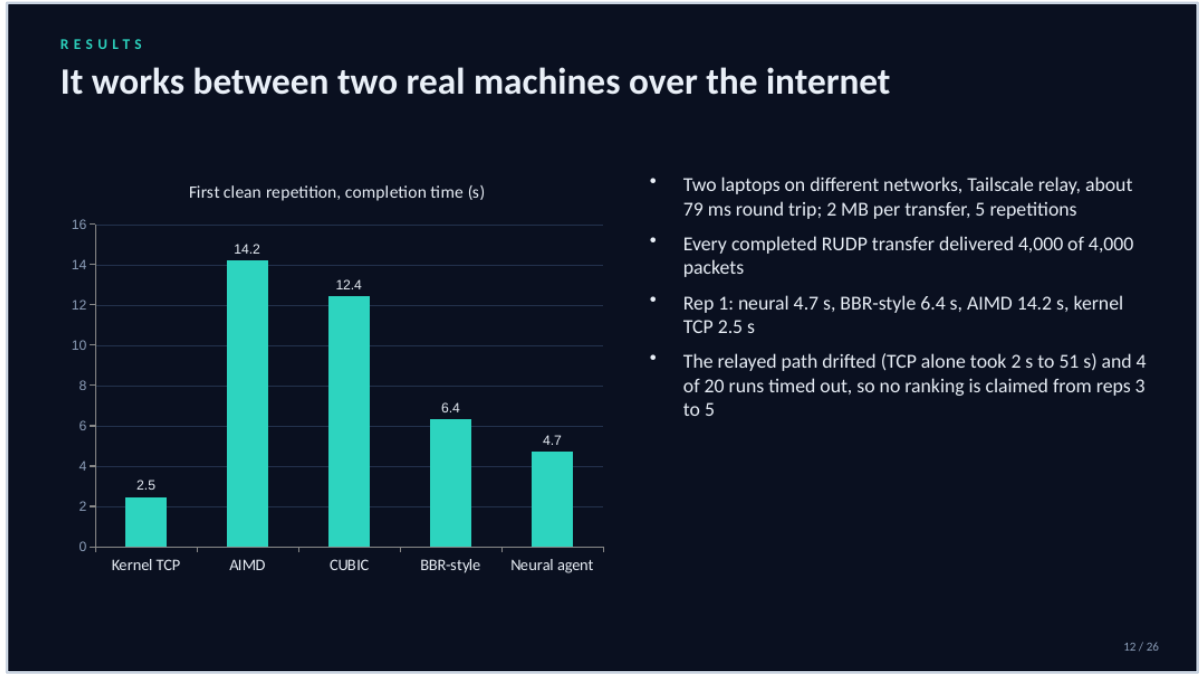
SAY

- Next to a kernel TCP flow the agent takes 24 to 34% and TCP keeps the rest. Jain fairness 0.84 against 0.64 for AIMD with unequal RTTs.

IF ASKED Is it polite? More pushy than AIMD; we have an optional polite mode.

12. Two machines

Time on this slide: 0:45



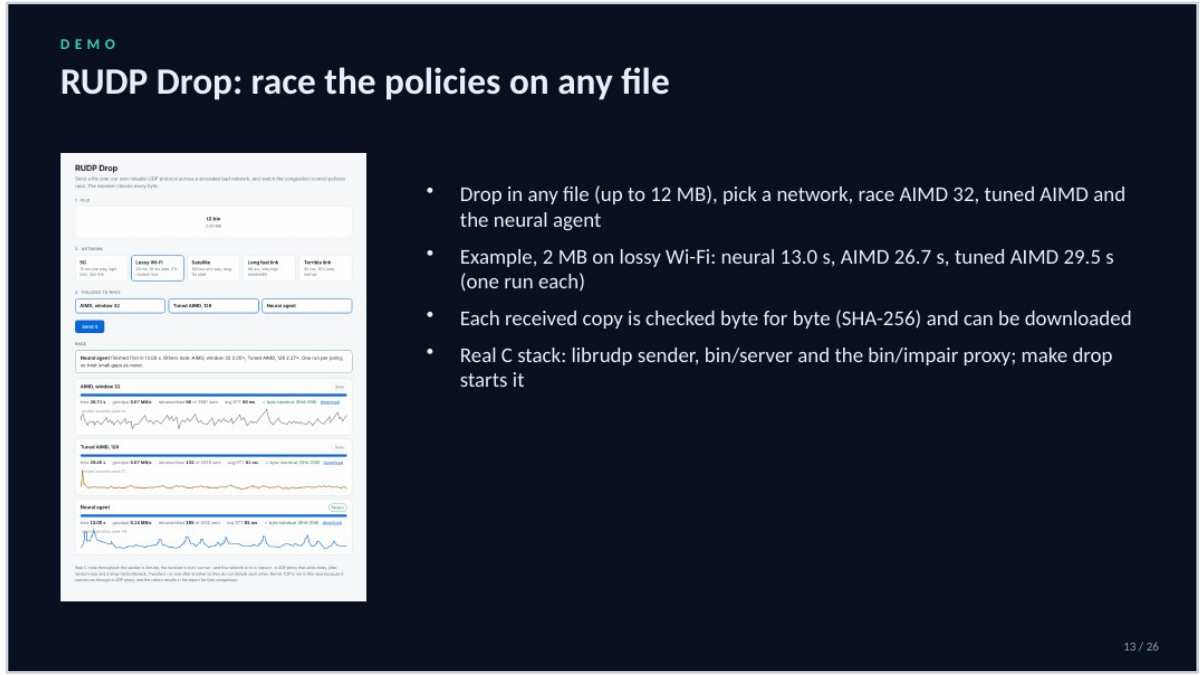
IF ASKED Why only one clean rep? The relayed path drifted; 4 of 20 runs timed out.

SAY

- Two laptops on different networks over Tailscale, 79 ms RTT. Every completed transfer delivered all 4000 packets; in rep 1 the agent took 4.7 s.
- The relay drifted, so no ranking from later reps.

13. RUDP Drop

Time on this slide: 1:00



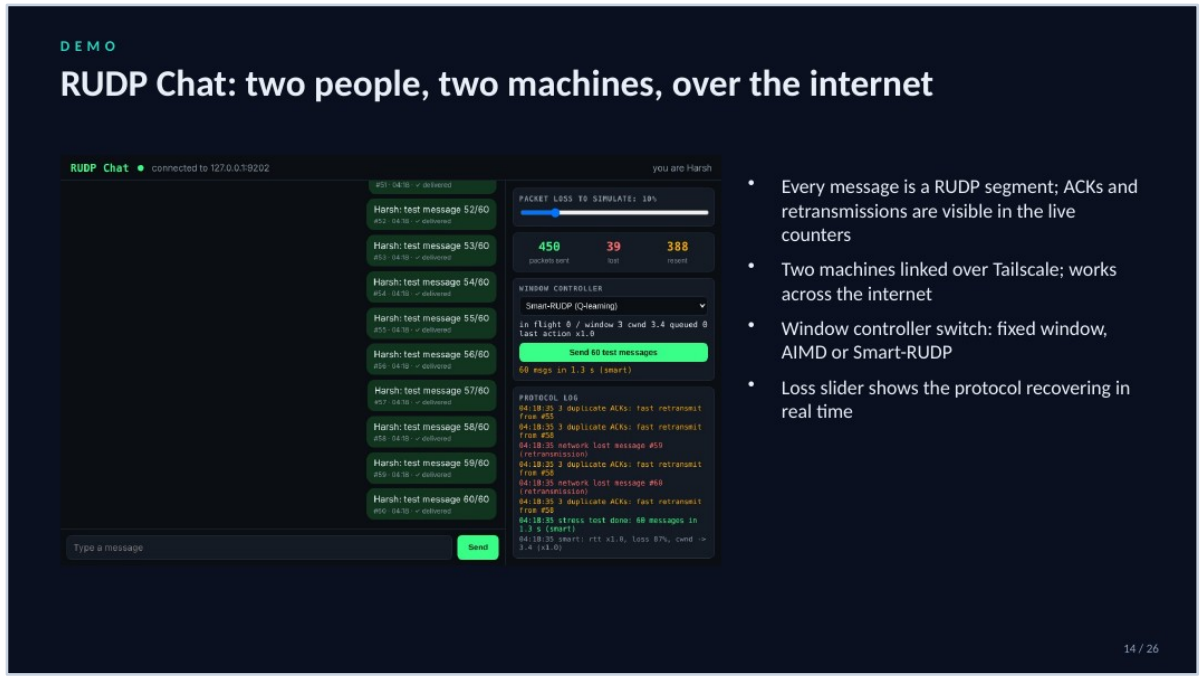
SAY

- Drop in a file, pick a network, race three policies. Example: 13.0 s neural against 26.7 s AIMD, byte-identical by SHA-256.

IF ASKED Real stack? Yes: C sender, server and impairment proxy.

14. Chat

Time on this slide: 1:00



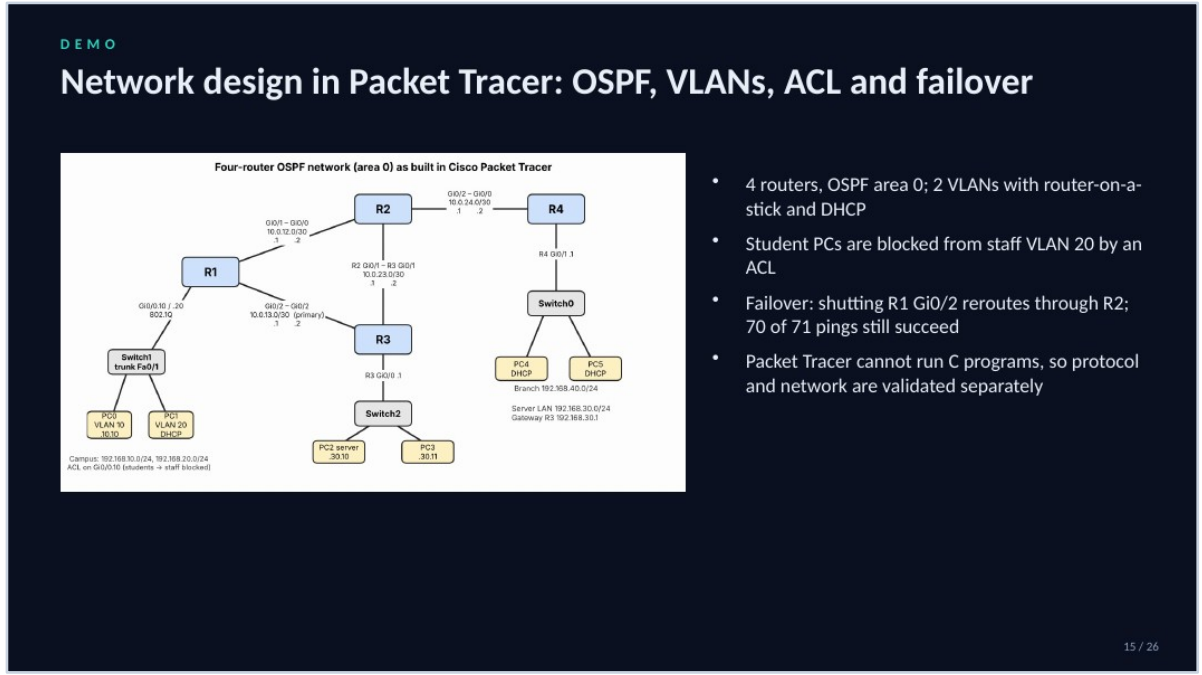
IF ASKED Does it run on the internet? Yes, linked over Tailscale.

SAY

- Chat over RUDP between two machines. Live counters show ACKs and retransmissions; the loss slider shows recovery.

15. Packet Tracer

Time on this slide: 0:50



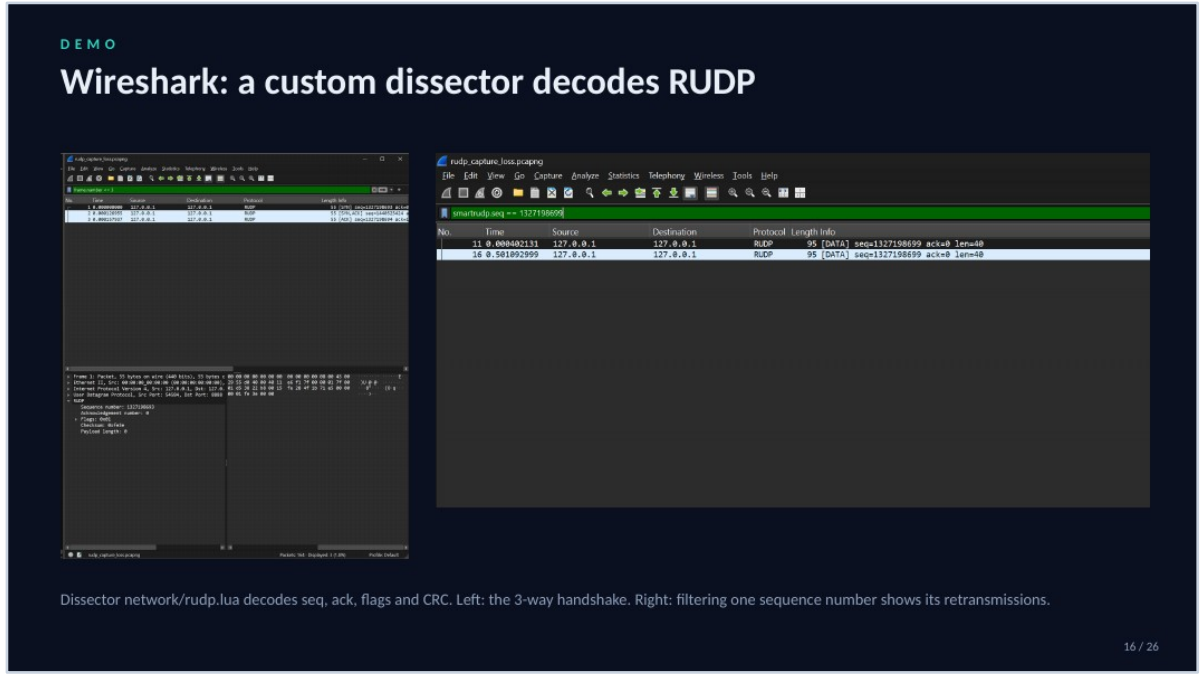
IF ASKED Why separate from the code? Packet Tracer cannot run C programs.

SAY

- Four routers, OSPF area 0, two VLANs, DHCP, an ACL, and failover: shutting R1 Gi0/2 reroutes through R2, 70 of 71 pings succeed.

16. Wireshark

Time on this slide: 0:40



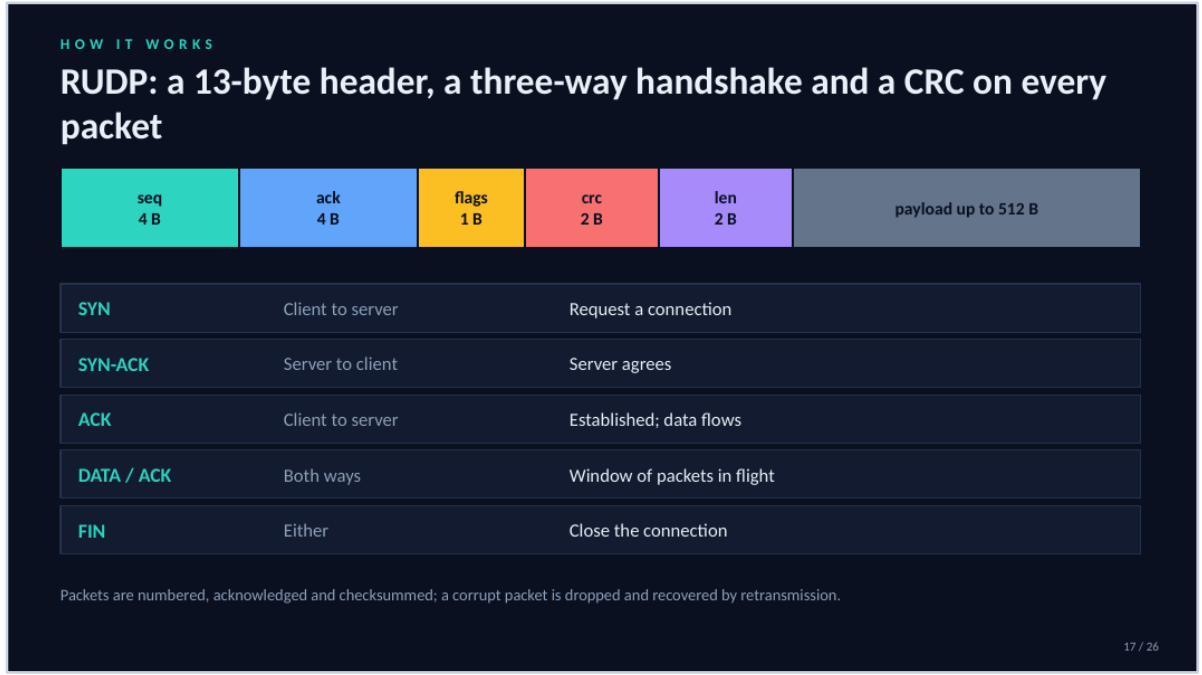
SAY

- Our Lua dissector decodes seq, ack, flags and CRC. Left, the handshake; right, filtering one sequence number shows retransmissions.

IF ASKED Is it a real capture? Yes, from the running protocol on loopback.

17. Protocol

Time on this slide: 0:50



SAY

- 13-byte header, three-way handshake, CRC-16 on every packet; corrupt packets are dropped and recovered.

IF ASKED Why CRC-16? A byte-sum accepted corrupt packets; CRC-16 accepted 0 of 333k.

18. Selective Repeat

Time on this slide: 0:50



IF ASKED Is the gain significant? All rows yes except hostile 10% loss.

SAY

- Receiver buffers out of order, 32-byte ACK bitmap, one timer per packet, adaptive timeout. About 10x fewer retransmissions than Go-Back-N.

19. Controllers

Time on this slide: 0:50



IF ASKED Why hand-over? Without it the agent was 7x slower at 10% loss.

SAY

- Five controllers behind one interface. The neural net is 5-32-32-5 with 1413 weights and five actions from 0.5x to 1.5x.
- AIMD takes over when an interval loses 10% or more.

20. Ablation

Time on this slide: 0:50



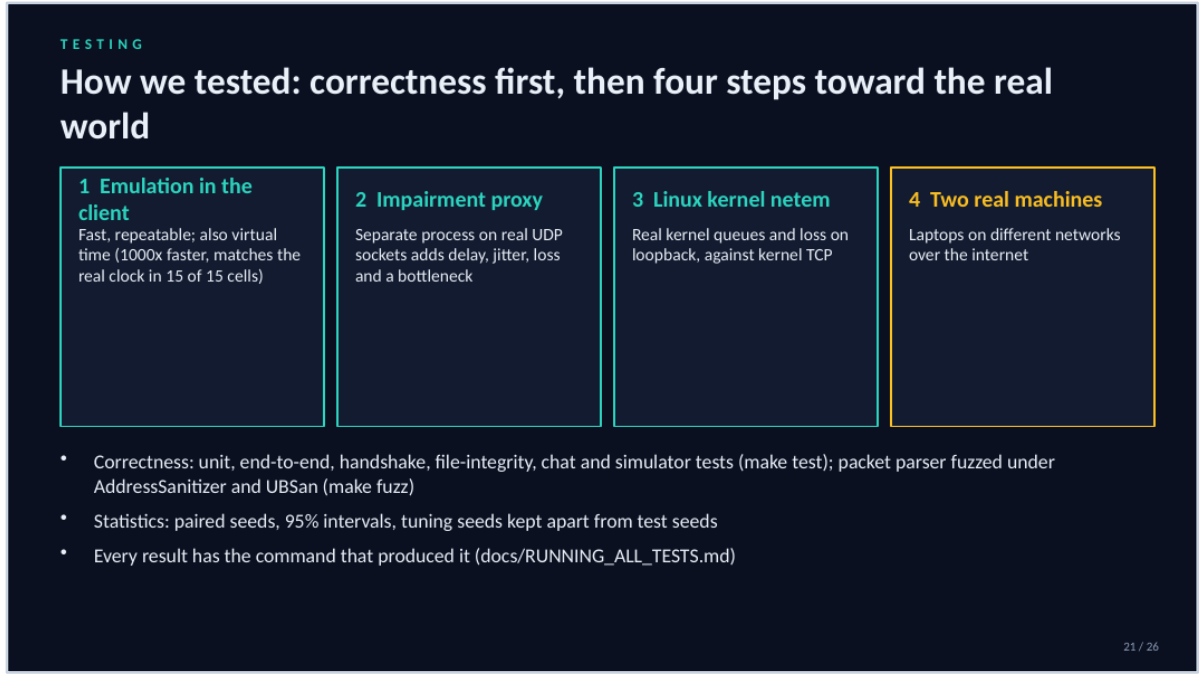
IF ASKED Did retraining help? Evolution strategy added only 2%, not adopted.

SAY

- On 100 random held-out paths the agent takes 0.53x the time. Cap 128 and original settings matter most.

21. How we tested

Time on this slide: 0:40



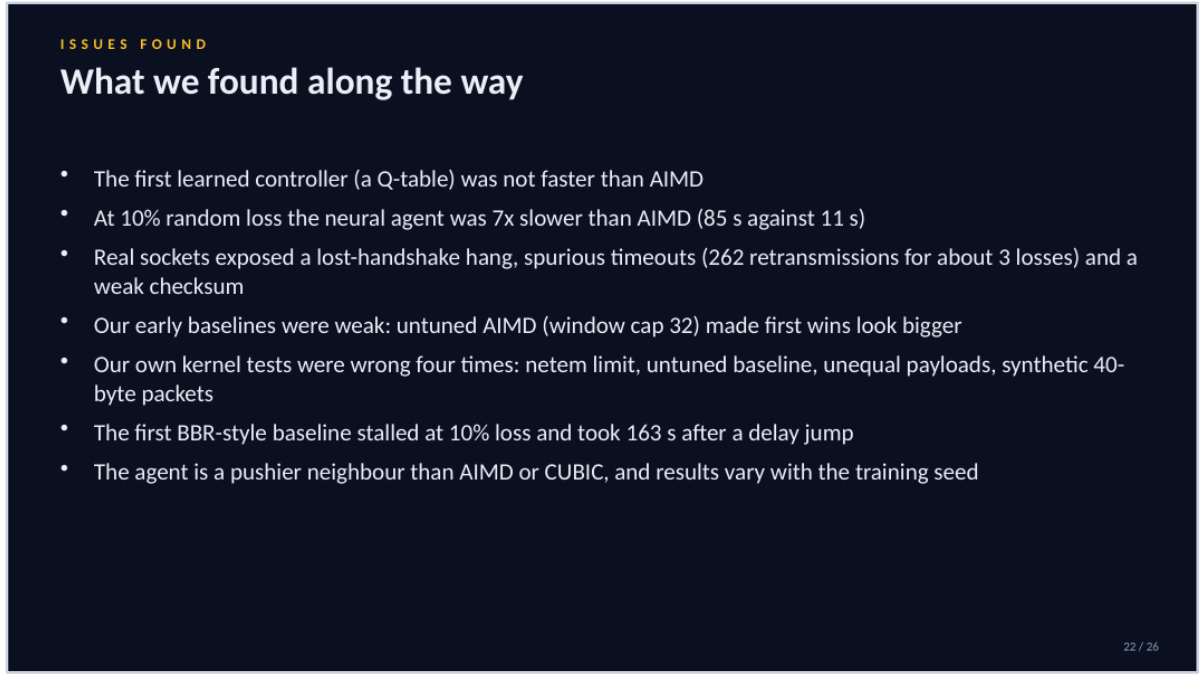
SAY

- Correctness first, then four steps toward the real world: emulation, impairment proxy, kernel netem, two machines.

IF ASKED Where are commands? docs/RUNNING_ALL_TESTS.md.

22. Issues

Time on this slide: 0:50



IF ASKED Were results hidden? No; all are in the report.

SAY

- What went wrong: the Q-table was not faster, 7x slower at 10% loss, socket bugs, weak early baselines, and four mistakes in our own kernel tests.

23. Changes

Time on this slide: 1:00



WHAT WE CHANGED

Every issue led to a fix

Before	After
Go-Back-N with a fixed 500 ms timer	Selective Repeat: bitmap ACKs, per-packet timers, adaptive timer with a 1.3 x srtt floor; about 10x fewer retransmissions
Byte-sum checksum	CRC-16: 0 of 333k corrupted packets accepted
Server ignored data after a lost final handshake ACK	Handled as in TCP, with a regression test
Q-table agent, window cap 32	5-32-32-5 neural network, cap 128, slow start, pacing
Neural agent 7x slower than AIMD at 10% loss	Hand-over to AIMD while loss is high: 13.9 s instead of 84.9 s
Untuned AIMD and CUBIC baselines	Tuned baselines, so every win is against a fair opponent
Faulty kernel and BBR tests	Netem limit, equal payloads and real 512-byte packets; kernel runs repeated; BBR stall and slow recovery fixed

23 / 26

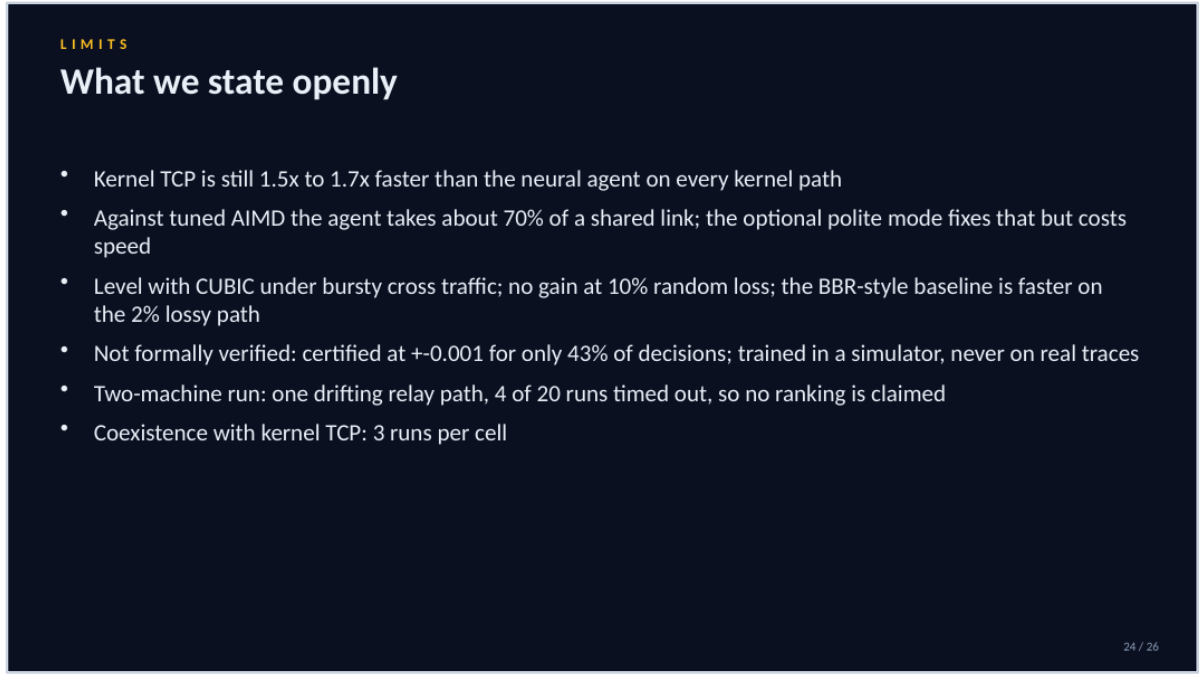
IF ASKED Biggest fix? Hand-over to AIMD: 84.9 s down to 13.9 s at 10% loss.

SAY

- Each issue led to a fix: Selective Repeat, CRC-16, tuned baselines, neural net with hand-over, corrected kernel tests.

24. Limits

Time on this slide: 0:50



IF ASKED Future work? Real traces, formal certification, a direct-LAN multi-machine run.

SAY

- Kernel TCP is faster; the agent is pushier than AIMD; no formal verification; trained in a simulator; two-machine run is one relayed path.

25. Learned

Time on this slide: 0:50



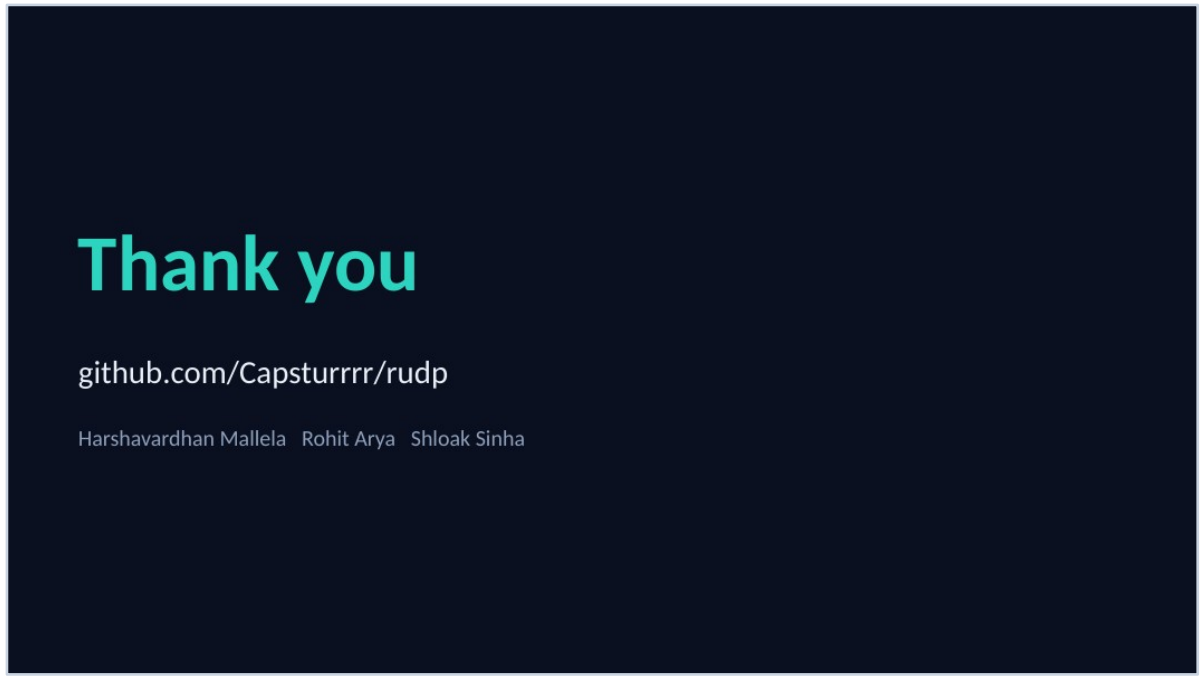
SAY

- We learned protocol design, honest evaluation, and that tuned baselines matter. Next: real traces and certification.

IF ASKED What would you do first? A bigger real-network study.

26. Thank you

Time on this slide: 0:15



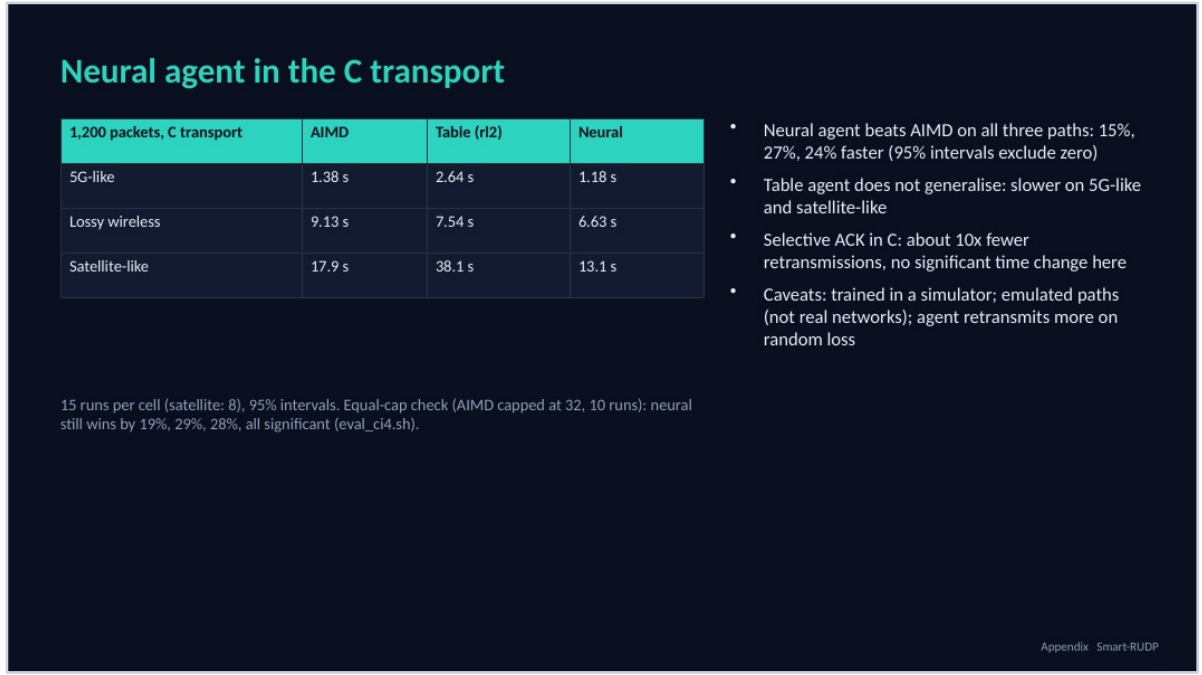
SAY

- Thank you. Code and results are at github.com/Capsturrrr/rudp.

IF ASKED Questions.

27. Neural agent in the C transport

Time on this slide: 1:00



IF ASKED Q: So Smart-RUDP now beats AIMD? A: The neural version does, in the C transport on emulated paths, trained in a simulator; the original table agent in Section 8 did not. Q: Is it fair? A: Same start window; AIMD keeps a cap of 128 while the agents are capped at 32; the agent retransmits more on random loss. Q: Why no gain from SACK? A: Those paths are limited by the bottleneck rate, so extra retransmissions cost little; the simulator, with different paths, showed a large gain.

SAY

- We then put the neural agent into the C transport, the same code as Section 8, and tested it on the same three emulated paths with 15 runs each.
- AIMD and the agents all start from a window of 10. The neural agent was faster than AIMD on all three: 15 percent on 5G-like, 27 percent on lossy, 24 percent on satellite-like. The intervals exclude zero.
- The retrained table did not carry over: it was slower on 5G and satellite. A network generalises to unseen paths where a table does not.
- We also added selective acknowledgments to the C code: about ten times fewer retransmissions, but no significant change in completion time on these emulated paths.
- We also reran it with AIMD capped at 32 like the agent: the neural agent still wins by 19, 29 and 28 percent, all significant.

28. Safety net: hand over to AIMD

Time on this slide: 0:50



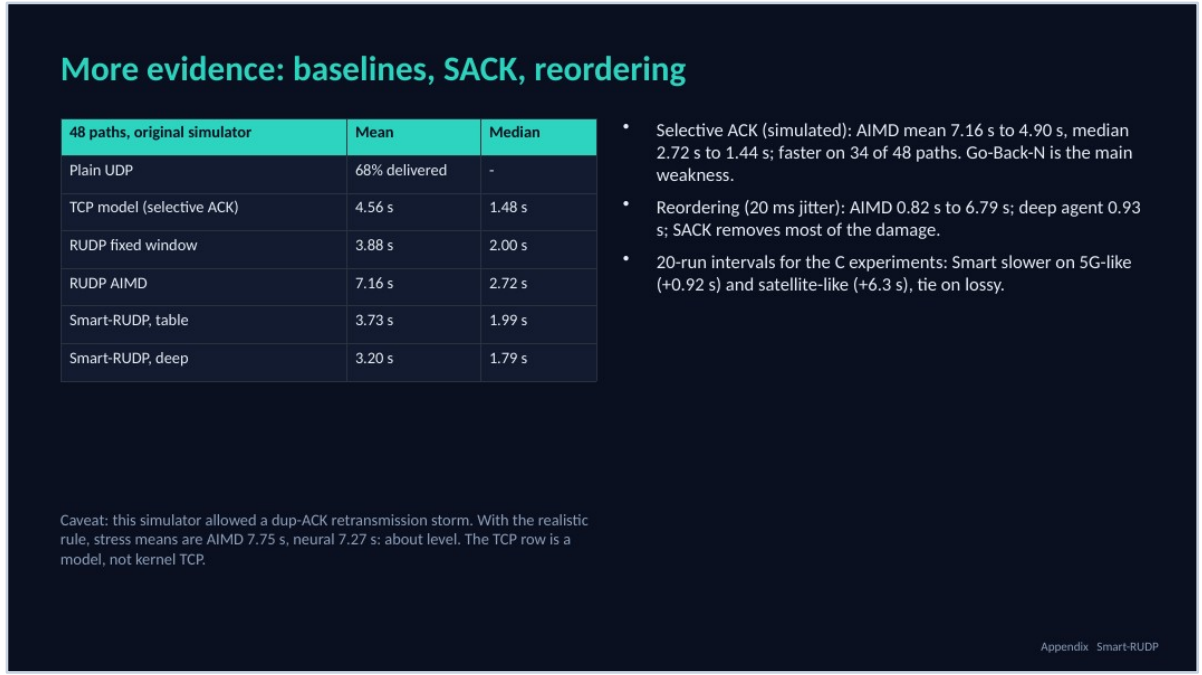
IF ASKED Is the hybrid still 'AI'? Yes on clean and moderate paths, where the neural network decides; on heavy loss it deliberately defers to AIMD. It is a rule we added, not something the network learned.

SAY

- While checking our own result we found where the neural agent fails: on a hostile path with 10 percent loss it needed about 85 seconds where AIMD needed 11.
- The fix is a hand-over: when a decision interval loses 10 percent or more of its packets, plain AIMD controls the window; after three clean intervals the agent takes over again.
- With that, the three benign paths keep the agent's advantage and the hostile path is back near AIMD at about 14 seconds.
- We also tried cutting the window on every timeout; it fixed the hostile case but made the 2 percent path much worse, so we dropped it. So the learned agent is useful on clean to moderate paths and needs a classical safety net elsewhere.

29. More evidence: baselines, SACK, reordering

Time on this slide: 1:00



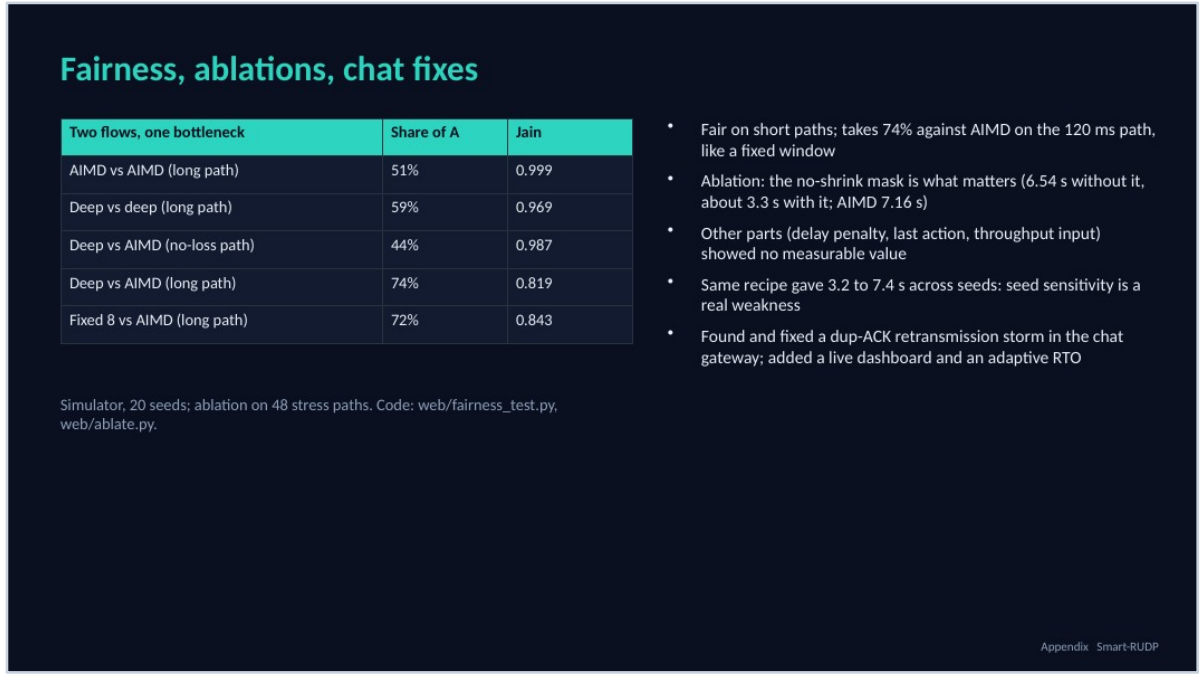
IF ASKED Q: Is the TCP row real TCP? A: No, it is a model with the same window cap; real kernel TCP needs netem on Linux, and we provide the script. Q: Why did you not implement SACK in C? A: Time; the simulation shows it is the biggest win and it is the first item of future work. Q: Is 20 runs enough? A: The intervals exclude zero for 5G and satellite and include zero for lossy. Did you check it is not a simulator artefact? Yes: with the realistic rule the simulated gain mostly disappears, so we lean on the C results, which use that rule and equal window caps.

SAY

- The abstract also promised TCP and plain UDP baselines, so we ran them on the same 48 unseen simulated paths.
- Plain UDP delivered only 68 percent of the messages, so reliability is needed. Our TCP model, with selective acknowledgments, has the lowest median at 1.48 seconds.
- Smart-RUDP deep is at 3.20 seconds mean and is about even with the TCP model path by path: faster on 22, slower on 23.
- The gap to TCP is mostly Go-Back-N. A selective-ack version of our transport cut AIMD from 7.16 to 4.90 seconds in simulation.
- Reordering hurts Go-Back-N badly: AIMD goes from 0.82 to 6.79 seconds at 20 milliseconds of jitter. The learned agents cope better and selective ack removes most of the damage.
- For the C experiments we repeated 20 runs per mode: Smart-RUDP is still slower on 5G and satellite and ties on lossy.
- Be upfront: our simulator originally allowed a duplicate-ACK retransmission storm that flatters aggressive controllers. With the realistic rule the neural agent is about level with AIMD in simulation (7.27 s against 7.75 s). The C results are the evidence we rely on.

30. Fairness, ablations, chat fixes

Time on this slide: 0:50



IF ASKED Is the agent fair? Mostly, but not on the long path against AIMD (74% share, Jain 0.82). Fairness was only tested in the simulator, not in C.

SAY

- We also tested two things people ask about. Fairness: two flows through one bottleneck. The neural agent is fair against itself and on short paths, but takes 74 percent against AIMD on the long 120 ms path, about like a fixed window.
- Ablation: we retrained the agent with one piece removed. Only the no-shrink mask matters; without it the agent is no better than AIMD. The other pieces showed no measurable value.
- The same recipe also gave very different results by seed, between 3.2 and 7.4 seconds, so we pick the seed on validation paths.
- While building the dashboard we found a retransmission storm in the web gateway and fixed it; there is also an adaptive timeout option now.

31. Fairness and file transfer

Time on this slide: 0:50



IF ASKED Would this be safe to deploy? Not on a shared link yet: it needs fairness-aware training and TCP coexistence tests. For a dedicated path it is a clear win.

SAY

- Fairness on real sockets: two neural flows split the link evenly, and so do two AIMD flows. But against tuned AIMD the neural agent takes 68 percent of the link, and 76 percent against the original AIMD. CUBIC takes 61 percent. So the agent is a more aggressive neighbour. Nothing starves, but it is not polite, and we did not test coexistence with TCP. That is our main limitation.
- We also added file transfer: the sender sends a real file and the received file is byte-identical in every run. A 4 megabyte file over the fast path takes 8.9 seconds with the agent against 21 seconds with tuned AIMD.
- The chat and web gateway now use the same window and bitmap as the C transport, and there is CI, fuzzing and an architecture document.

32. Closing gaps: fairness and cross traffic

Time on this slide: 0:50



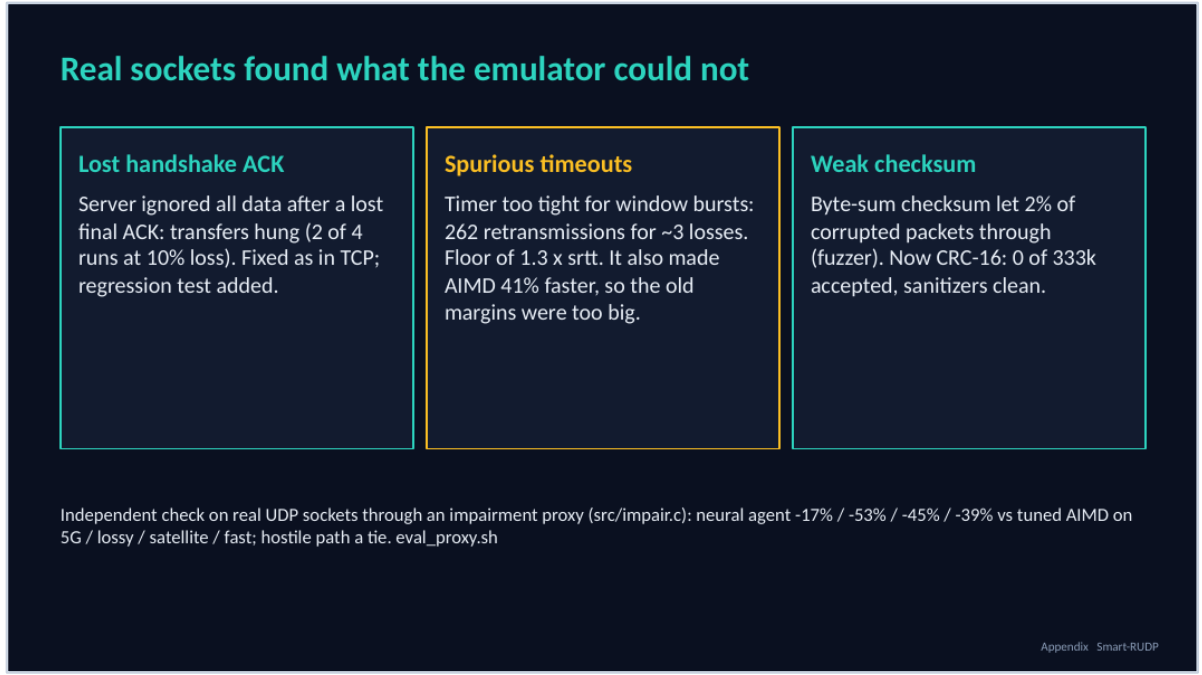
IF ASKED Is polite mode learned? No. It is a hand-made rule on the delay signal. A learned version needs a two-flow training environment, which is future work.

SAY

- We worked on the weaknesses we had listed. For fairness we added an optional polite mode: when a standing queue builds, the agent backs off. Over 8 runs against tuned AIMD it takes 52 percent of the link instead of 70, and the Jain index rises from 0.86 to 0.94. The price is speed: on 100 random paths the average time goes from 0.53 to 0.58 times AIMD's, and the worst path is 2.95 times slower, so it stays off by default.
- We also added bursty cross traffic to the test proxy. Here the result is less flattering: the agent is 15 percent faster than tuned AIMD, but level with CUBIC, and it retransmits about three times as often as AIMD. Three tweaks did not fix that, so we report it as a limit. Finally the server now handles up to 16 clients, checked by sending two files at once.

33. Real sockets found what the emulator could not

Time on this slide: 0:50



IF ASKED Is this a real network? It is real UDP sockets on one machine, with the impairment applied by a separate process. It is not a kernel netem or a multi-machine path; we provide the script for that but had no root access.

SAY

- The emulator is inside our client, so we also tested through a separate impairment process on real UDP sockets. That found three real defects.
- First, if the client's last handshake ACK is lost, the server ignored all following data and the transfer hung. We fixed it the way TCP does and added a regression test. Second, the spurious timeouts I just mentioned. Third, a fuzzer showed our byte-sum checksum let about 2 percent of corrupted packets through, so it is now CRC-16 and the fuzzer accepts none.
- On real sockets the neural agent is again fastest on every path: 17, 53, 45 and 39 percent faster than tuned AIMD on 5G, lossy, satellite and fast, and a tie on the hostile path.

34. Problems we hit and fixed

Time on this slide: 0:40



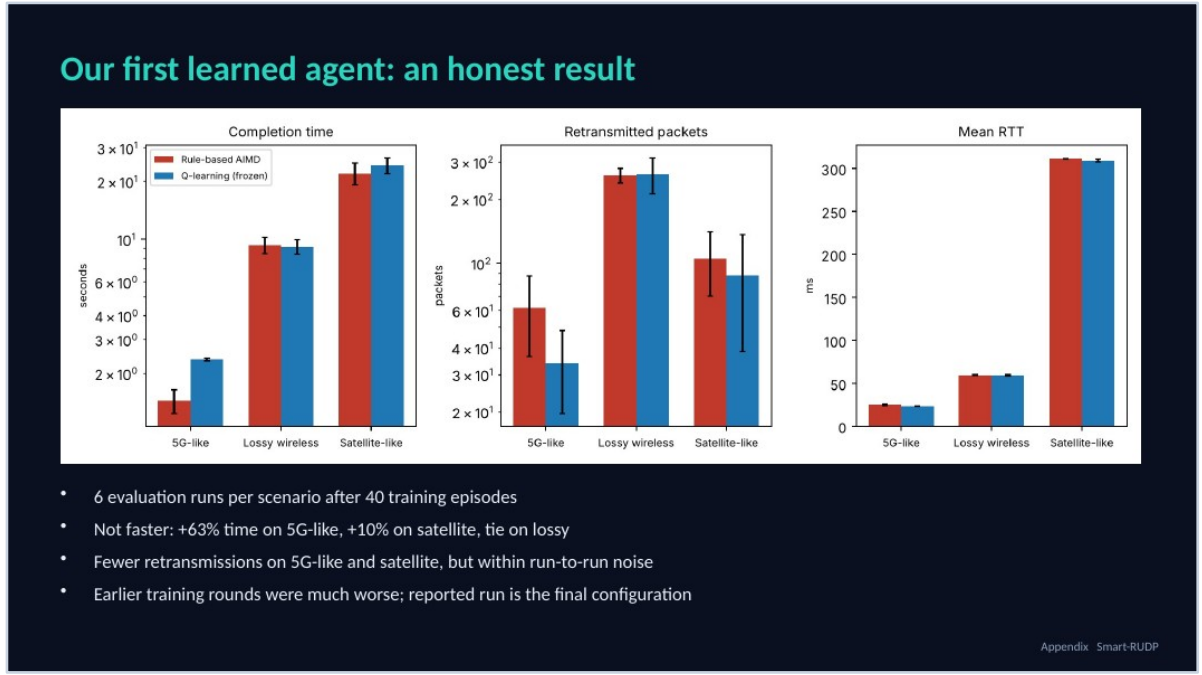
IF ASKED Say these quickly: it shows real debugging, which is what separates a project from a lab.

SAY

- Server stuck after a killed client: it hung 15.5 s; fix was to accept SYN in any state.
- A leftover netem rule distorted results; we now check tc qdisc before each test.
- Windows Wireshark saw nothing: capture inside WSL.
- Benchmark script portability: replaced date and grep tricks with awk and sed.

35. Our first learned agent: an honest result

Time on this slide: 0:50



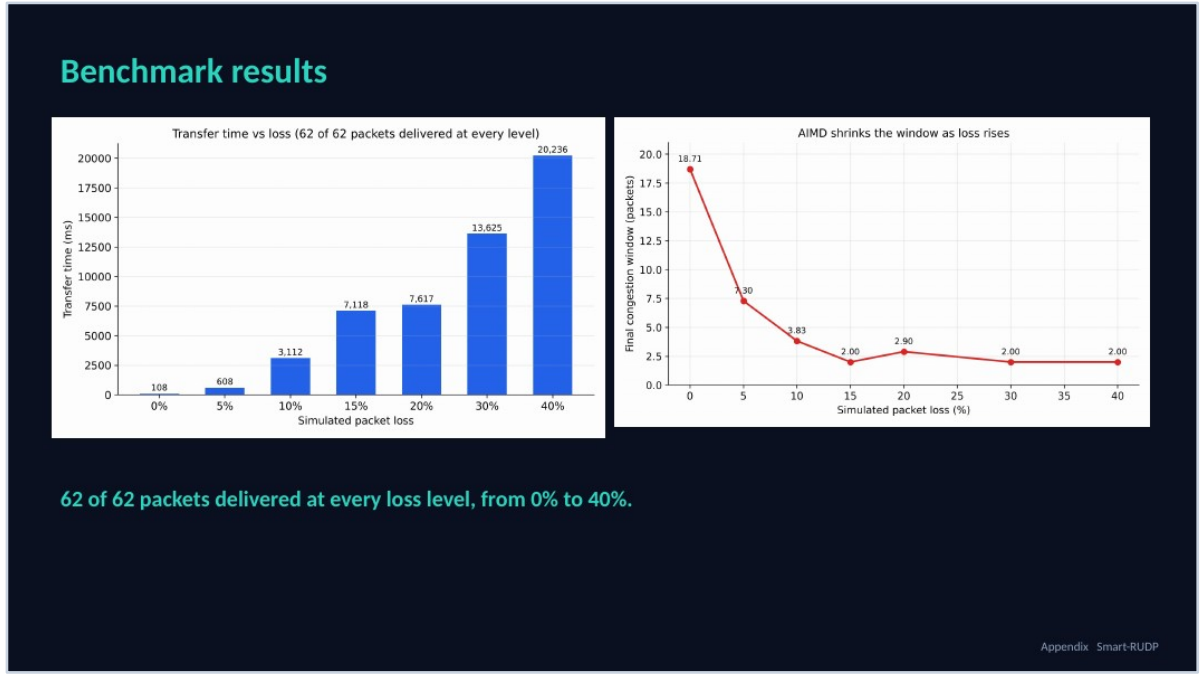
IF ASKED Q: Then what was the point? A: The experiment. We tested whether a learned window helps and report the answer, including the negative result.

SAY

- Be honest: it did not beat AIMD. Plus 63 percent time on 5G-like, plus 10 percent on satellite, a tie on the lossy path.
- It used somewhat fewer retransmissions on two paths, within run-to-run noise.
- Earlier training rounds were much worse; this is the final configuration. The finding is that a simple learner does not beat a tuned rule.
- In the chat, retrained for bursty random-loss traffic, it does beat AIMD on most simulated paths; mention that on slide 19.

36. Benchmark results

Time on this slide: 0:40



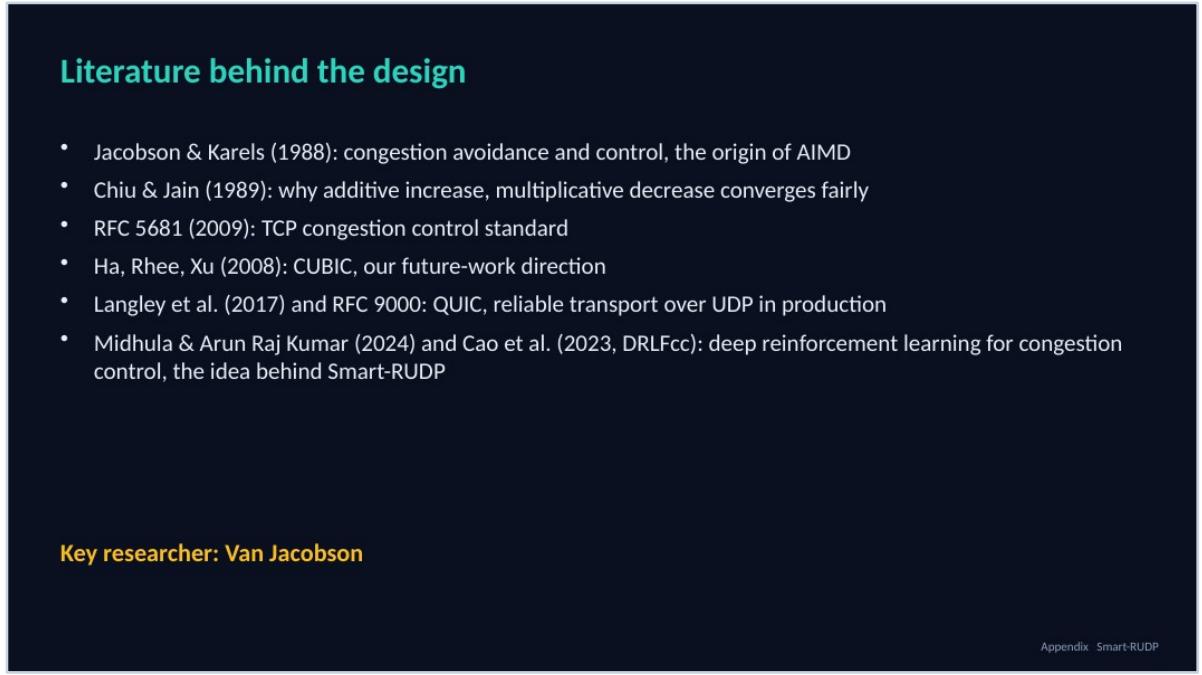
IF ASKED Q: Single runs? A: Yes, the original benchmark ran each level once. Smart-RUDP evaluation uses 6 runs per scenario.

SAY

- 62 of 62 packets were delivered at every loss level from 0 to 40 percent.
- Higher loss costs time and retransmissions, never correctness.

37. Literature behind the design

Time on this slide: 0:30



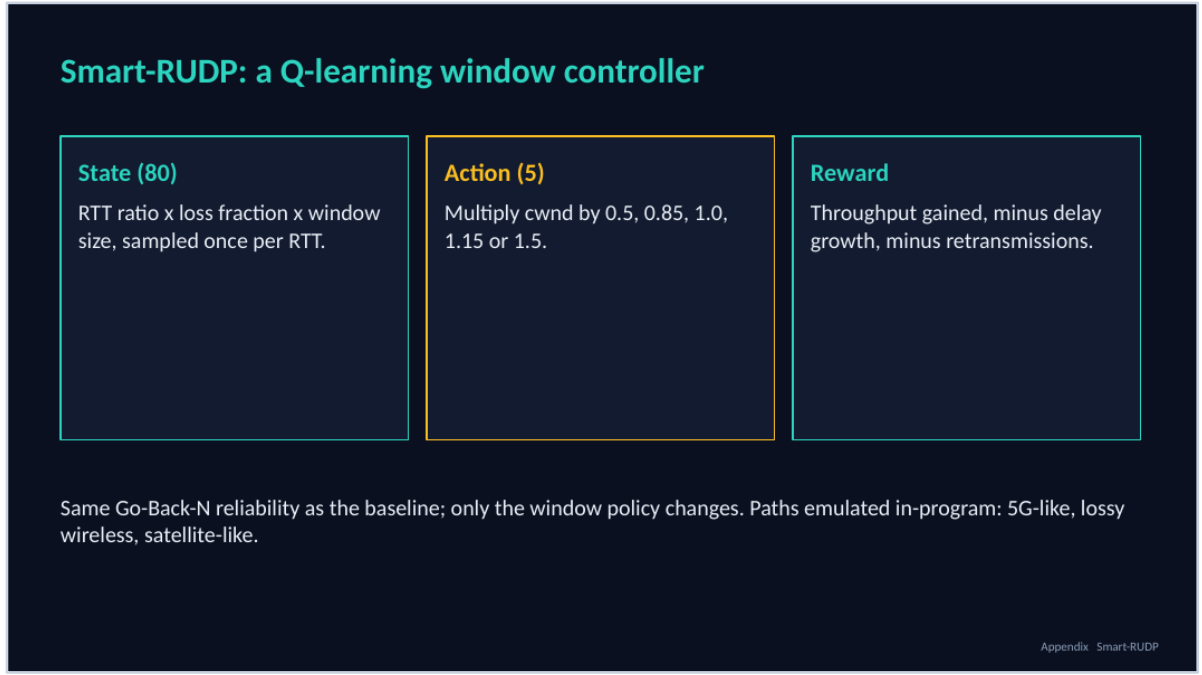
IF ASKED Q: Why AIMD? A: Chiu and Jain show it converges to a fair share. Q: Are those papers deep RL and yours is not? A: Yes; we used a lookup table, which is simpler and inspectable.

SAY

- Jacobson and Karels 1988 is the origin of AIMD; Chiu and Jain 1989 explains why it converges fairly.
- RFC 5681 is the standard; CUBIC is future work; QUIC and RFC 9000 are reliable transport over UDP in production.
- The AI part comes from Midhula and Arun Raj Kumar (2024) and Cao et al. (2023, DRLFcc): deep reinforcement learning for congestion control. Smart-RUDP applies the same idea with a smaller tabular Q-learning agent.
- Key researcher: Van Jacobson.

38. Smart-RUDP: a Q-learning window controller

Time on this slide: 0:50



IF ASKED Q: Is it really learning? A: Yes, 40 training episodes; the learning-curve figure shows it. It is small and tabular with limited training.

SAY

- Q-learning with 80 states (RTT ratio, loss fraction, window size), sampled once per RTT.
- Five actions: multiply the window by 0.5, 0.85, 1.0, 1.15 or 1.5.
- Reward: throughput gained, minus delay growth, minus retransmissions. Same Go-Back-N reliability; only the window policy changes. Paths are emulated in the program: 5G-like, lossy wireless, satellite-like.

39. The AI agent in our code

Time on this slide: 1:00



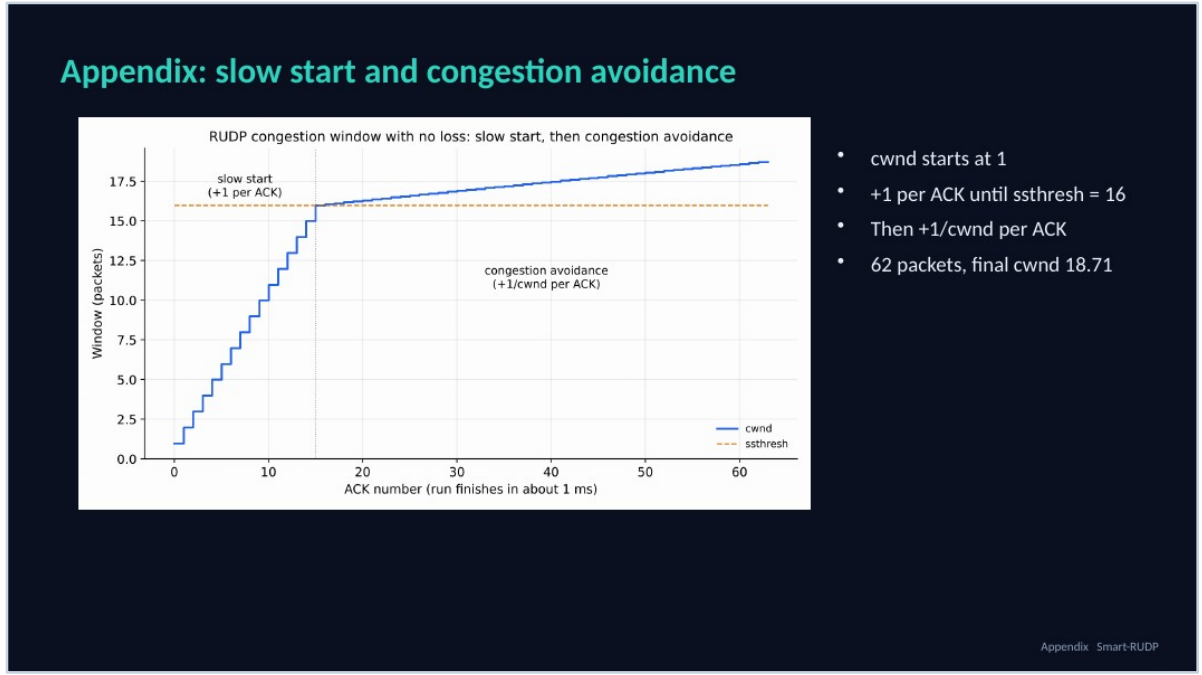
IF ASKED Q: Is this deep RL? A: The C agent is tabular. The chat also has a deep agent, a small DQN in numpy (5-32-32-5). Q: Does deep beat the table? A: Modestly: 3.20 s vs 3.73 s mean on 48 unseen paths, faster on 19, slower on 6. Q: Any problem? A: Training is seed-sensitive: 1 of 4 seeds failed, so we select on validation paths. It is simulated and not in the C code. Q: Where is it trained? A: web/train.py for the chat, the smart client trains online for the main experiments; table in results/q_chat.txt.

SAY

- This is the agent itself. Left: rl_cc.h, 73 lines of C. The state is 80 buckets of RTT ratio, loss and window size.
- choose picks one of five window multipliers, usually the best, sometimes random so it keeps exploring.
- update is the Q-learning rule: move the value of the action we took toward the reward plus the best value of the next state.
- Right: the loop in smart_client.c. Once per RTT it computes the reward, throughput minus delay growth minus loss, updates the table, picks the next action and scales the window.
- The code on the slide is simplified; the real files are in the repo.
- We also built a deep version: web/dqn.py is a small neural network, two hidden layers of 32, trained with DQN in the same simulator. It reads continuous inputs instead of buckets.

40. Appendix: slow start and congestion avoidance

Time on this slide: -



IF ASKED Use this slide to answer 'show me congestion avoidance'.

SAY

- Only if asked: the window starts at 1, adds 1 per ACK until the threshold 16, then 1/window per ACK. In the 62-packet run the final window was 18.71.

Hard questions and short answers (1 of 3)

Is this a lab exercise?

A lab sends datagrams. We built reliability and congestion control on top, measured them, and used them in a real chat and a routed network.

Why not TCP?

TCP is in the kernel and cannot be changed. Building our own makes every mechanism visible. QUIC and HTTP/3 do the same.

Did Smart-RUDP win?

Yes, in the final C transport against tuned baselines: 15 to 52 percent less time than tuned AIMD on four ordinary paths (real clock, real sockets and 100 random held-out paths agree), a tie on the 10 percent loss path. Against a simplified BBR it wins on the satellite-like and fast paths and loses on the 2 percent lossy path. The first tabular agent did not beat AIMD, and we say so.

Why Go-Back-N?

We started with it because it is simple to build and explain, then replaced it as the default with Selective Repeat: the receiver buffers out-of-order packets and reports them in an ACK bitmap, and the sender resends only what is missing. Go-Back-N stays behind a flag for comparison.

Was it tested between machines?

Yes. Delay, loss and jitter are emulated in our client, in a separate proxy on real UDP sockets, and in virtual time; these agree with each other. We also ran it once on the Linux kernel with tc netem on one laptop: the agent beat tuned AIMD there, but kernel TCP was 1.5 to 1.7 times faster than us. We also made one run between two laptops on different networks over a Tailscale relay: RUDP delivered every transfer that finished, but the path drifted and four of twenty runs timed out, so we do not rank the policies from it; kernel TCP was 2 to 6 times faster on the clean repetitions.

Hard questions and short answers (2 of 3)

What does Packet Tracer prove?

The network design: OSPF, VLANs, ACL, 70/71 failover. The protocol is validated separately with emulation and Wireshark.

Peer disappears?

Sender keeps retransmitting on timeout. A connection abort after repeated timeouts is not built; it is the fix we would add.

Does the checksum catch errors?

It is CRC-16/CCITT now. The first version was a byte sum and a fuzzer showed 2 percent of corrupted packets got through; with CRC-16 none of 333,000 did. It cannot catch every error pattern, as no 16-bit check can.

Why no handshake in the exported trace?

The export builds synthetic packets. The real tshark capture shows SYN, SYN-ACK, ACK.

Is the baseline fair?

We tuned it: window cap 128 and a high start threshold, plus the same retransmission-timer floor for every policy. We also compare against tuned CUBIC. Tuned AIMD improved a lot, and the agent is still ahead on four paths.

Hard questions and short answers (3 of 3)

Is it fair to other flows?

No, not entirely: against tuned AIMD it takes 68 percent of a shared link, (CUBIC takes 61 percent). Nothing starves. We did not train for fairness. With unequal round-trip times two neural flows are fairer than two AIMD flows (Jain 0.84 against 0.64 at +150 ms). Against a kernel TCP flow on the same link it takes only 24 to 34 percent, so it does not starve TCP (three runs per cell).

How do you know the simulator is right?

The virtual-time mode runs our real C sender code with a simulated receiver and clock. It matched the real-clock runs in all 15 cells, and the real-socket proxy agrees too.

Is Q-learning really learning?

80 states, 5 actions, 40 episodes; the learning-curve figure shows it. Small and tabular, with limited training.

Did you compare with BBR?

Yes, a simplified BBR-style policy, not the Linux one. It beats tuned AIMD on four of five paths, is faster than our agent on the 2 percent lossy path, and slower on the satellite-like and fast paths. Two bugs in our own version were found and fixed.

Can the neural network be trusted?

In practice it is tolerant: noise of 0.2 on its inputs costs at most 12 percent, and a small decision tree gives the same transfer time on four of five paths. It is not formally verified: certification works only at tiny input changes.